SMART: Scheduling of Multi-Agent RTL Trajectories via EDA-Informed Online Decision Making

Anonymous Author(s)
Affiliation
Address
email

Abstract

Recent agentic register-transfer level (RTL) generation systems improve functional correctness by scaling test-time compute through multi-agent loops that repeatedly generate, evaluate, and revise candidates using feedback from EDA tools. However, most systems use static schedules with a fixed number of parallel candidates and a fixed iteration depth. Such schedules allocate work without considering task difficulty or observed progress, leading to redundant candidate generation and EDA evaluations. They also lack an explicit mechanism for leaving a regressed trajectory and returning to an earlier viable candidate. This inefficiency is especially costly in industrial RTL development, where each evaluation consumes licensed EDA capacity and may incur long queueing and simulation delays. We formulate test-time compute scaling for multi-agent RTL generation as EDA-informed online scheduling over RTL design trajectories. We present SMART (*Scheduling of Multi-Agent RTL Trajectories*), which updates the schedule after each EDA evaluation by deciding which trajectories to initiate, continue, compare, revisit, or terminate. SMART retains earlier candidates and associates each candidate with its own evaluation evidence, enabling recovery from functional regression without mixing debugging contexts across trajectories. On the CVDP agentic RTL generation subset and the IP-level ChipVerilog benchmark, SMART achieves pass rates comparable to ACE-RTL while reducing completed EDA evaluations by 40.1% and 80.5%, respectively. These results demonstrate that EDA-informed online scheduling can maintain functional performance while substantially reducing the EDA evaluations required for test-time scaling.

1 Introduction

Agentic RTL generation has moved beyond one-shot code completion toward workflows that combine language models with executable EDA feedback. AutoChip closes the loop between RTL generation, compilation, and simulation [Thakur et al., 2023], while VerilogCoder adds planning and waveform tracing [Ho et al., 2024]. More recent systems divide planning, generation, checking, and debugging among specialized agents. MAGE, RTLSquad, and ACE-RTL follow this multi-agent design with different coordination and context-management strategies [Zhao et al., 2024, Wang et al., 2025, Deng et al., 2026], while HORIZON studies evaluator-driven hardware evolution over longer trajectories [Yu et al., 2026]. Together, these systems make multi-agent loops a practical pattern for generating and revising RTL against tool feedback.

Recent systems also improve functional results by scaling the computation used after a specification is given. MAGE [Zhao et al., 2024] increases candidate diversity before targeted debugging, ACE-RTL [Deng et al., 2026] runs multiple iterative trajectories, and HORIZON [Yu et al., 2026] extends evaluator-driven iteration over a longer horizon. Their reported gains come from more parallel

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

candidates, more EDA-feedback-driven iterations, or both. Test-time compute scaling therefore provides a clear way to improve agentic RTL generation. It does not, however, determine how that compute should be allocated after task-specific evidence becomes available.

Most existing agentic RTL frameworks follow a prespecified workflow, with both the number of parallel candidates and the number of iterative steps fixed before generation begins. Such schedules do not adapt to task difficulty or observed progress, so simple designs may receive unnecessary candidate generation while difficult designs remain confined to unproductive iterations. In addition, refining each candidate along an isolated trajectory prevents useful findings from being shared across trajectories and offers no explicit way to return to an earlier viable candidate after a functional regression. More importantly, costly EDA feedback turns this inefficiency into a barrier to deployment. In real RTL projects, compilation, simulation, and regression rely on commercial EDA licenses and often have long turnaround times, particularly when licenses and compute infrastructure are shared. Repeated tool calls for low-value iterations consume verification capacity and slow the overall RTL development cycle. This overhead can outweigh the benefit of additional generation and iteration and limit the use of agentic RTL generation in practice. Effective test-time scaling therefore requires adaptive resource allocation, evidence sharing across trajectories, and the ability to revisit earlier candidates rather than more iterations alone.

To address these challenges, we formulate test-time scaling as EDA-informed online scheduling over multi-agent RTL trajectories. Each EDA evaluation assesses the current candidate and provides evidence for allocating subsequent agent work. The resulting schedule adapts both the number of active trajectories and their iteration depths to task-specific progress instead of committing to a fixed allocation before generation. Candidate-specific evidence allows useful findings to be reused across trajectories without merging unrelated debugging contexts, while retaining earlier candidates enables explicit recovery from functional regression. EDA feedback therefore determines how RTL is revised and where additional test-time compute is spent.

We propose SMART, an EDA-informed multi-agent RTL generation framework centered on an adaptive trajectory scheduler. The scheduler coordinates planning, RTL generation, feedback review, and conditional trajectory comparison around the native EDA evaluator. After each evaluation, the feedback reviewer converts compiler and simulator outputs into candidate-specific evidence. When trajectories represent competing implementation hypotheses, the trajectory comparator consolidates their evidence before further work is allocated. The scheduler then decides which trajectories to initiate or continue, whether to revisit an earlier candidate, and when to select a candidate that has passed the evaluator and stop. Because candidates and their evidence remain separated by trajectory, SMART can reuse observations without accumulating a single mixed debugging history. Together, these mechanisms replace a prespecified multi-agent loop with task-dependent decisions over trajectory count, iteration depth, and recovery. Our contributions are summarized as follows:

- We formulate test-time scaling for multi-agent RTL generation as EDA-informed online trajectory scheduling, making trajectory allocation and recovery decisions after each EDA evaluation rather than fixing them before generation (Section 3).
- We propose SMART, which combines an adaptive scheduler with candidate-specific EDA evidence, conditional trajectory comparison, and retained candidates to adapt trajectory count and iteration depth and recover from functional regression (Section 4).
- Across CVDP and ChipVerilog, SMART achieves pass rates comparable to ACE-RTL while reducing completed EDA evaluations by 40.1% and 80.5%, respectively. (Section 5).

2 Related Work

2.1 Multi-agent RTL generation with EDA feedback

Recent agentic RTL systems combine LLM-based generation with compiler and simulator feedback. AutoChip and VerilogCoder establish iterative generation and debugging loops, while MAGE, RTLSquad, and ACE-RTL divide planning, generation, checking, and repair among specialized roles or parallel trajectories [Thakur et al., 2023, Ho et al., 2024, Zhao et al., 2024, Wang et al., 2025, Deng et al., 2026]. HORIZON extends evaluator-driven iteration to repository-level code evolution, whereas ChipCraftBrain adapts agents and generation settings within a validation-first pipeline [Yu et al., 2026, Eryilmaz, 2026]. These systems show the value of executable feedback and additional

test-time compute, but their execution schedules are typically configured in advance or iteration proceeds along isolated trajectories. Feedback may guide the next edit without deciding how many alternatives to open, which trajectory to prioritize, or when to return to an earlier candidate.

SMART targets this scheduling gap. It makes candidate selection, the number of concurrent trajectories, iteration depth, trajectory comparison, returning to earlier candidates, and stopping online decisions. Retained candidates and candidate-specific evidence allow later rounds to recover from regression and reuse observations across trajectories. The distinction is candidate-centric: rather than only changing an agent configuration or extending the current implementation, SMART schedules work over multiple recoverable RTL candidates.

2.2 Adaptive test-time compute and workflow control

Snell et al. [2025] studies the allocation of test-time compute and shows that the best strategy depends on prompt difficulty. This finding motivates adaptive rather than fixed scheduling in SMART. For RTL generation, scheduling governs parallel candidate generation and iterative refinement, while executable feedback arrives after every evaluated candidate. Each compiler or simulator result can therefore change which trajectory deserves further work. We make scheduling part of the EDA-feedback loop and report realized candidate and EDA counts without treating the scheduler as a compute-optimal policy.

General workflow research adapts computation at several levels. AFlow searches over workflows offline; MaAS and difficulty-aware orchestration select query-dependent agent configurations; Flow and Magentic-One adjust subtask execution or replan during a run [Zhang et al., 2025b,a, Su et al., 2025, Niu et al., 2025, Fourney et al., 2024]. Progress- and cost-aware routing further conditions model or agent selection on estimated gain, resource use, or stopping decisions [Li et al., 2026, Wang et al., 2026, Fang et al., 2026]. These methods provide close precedents for adaptive control, but their primary decision unit is a model, agent, operator, or workflow step. SMART instead decides which retained RTL candidate should receive the next EDA-backed action and whether to start an alternative trajectory, compare trajectories, return to an earlier candidate, or stop. This distinction matters because each observation belongs to a particular candidate, and recovery requires returning to that candidate.

2.3 Conditional multi-agent collaboration

Multi-agent discussion can resolve conflicting hypotheses, but mandatory debate adds communication cost. ReConcile uses repeated discussion, while DOWN activates debate conditionally and Optima studies communication efficiency [Chen et al., 2023, Eo et al., 2025, Chen et al., 2024]. SMART invokes its trajectory comparator only when named trajectories contain conflicting evidence. This shares diagnoses without adding a fixed debate stage; the trajectory scheduler still decides whether to compare trajectories, continue evaluation, return to an earlier candidate, or stop.

3 Problem Formulation

Multi-agent RTL generation repeatedly produces candidates, evaluates them with EDA tools, and uses the resulting feedback to decide what to try next. Existing workflows typically fix the number of parallel candidates and iterative rounds before this feedback is observed. Our question is how test-time scaling can allocate work among RTL design trajectories according to observed EDA feedback, so that viable trajectories receive sufficient iteration without wasting candidate generation and EDA evaluations. We therefore formulate test-time compute allocation as online scheduling over RTL design trajectories.

3.1 Candidate and evidence records for scheduling

Allocating test-time compute requires a record of which RTL implementations have been tried, how they are related, and what each evaluation revealed. Given a specification S, we call each generated implementation an *RTL candidate*. Evaluating a candidate produces compiler and simulator outputs, which we refer to as its *EDA evidence*. Continuing from a candidate produces its next iteration, whereas starting from the specification or an earlier candidate can introduce an alternative

implementation. We use $\mathcal{H}_r$ to denote the resulting candidate history at round r, including each retained candidate, its predecessor, and the decision that produced it. Following predecessor links identifies an *RTL design trajectory*; retaining multiple successors preserves alternative trajectories.

We define the scheduling context as $\mathcal{S}_r = (\mathcal{H}_r, \mathcal{E}_r)$, where $\mathcal{E}_r$ associates every evaluated candidate with its own EDA evidence. The history records the available candidates and their relationships, while the candidate-indexed evidence records how each one performed. Together, they allow the scheduler to compare trajectories and allocate further test-time compute without mixing feedback from different implementations.

3.2 Scheduling test-time compute over trajectories

Given the specification and the current scheduling context, a policy selects a variable-length set of decisions for the next round,

$$\mathcal{A}_r \;=\; \pi(S,\, \mathcal{S}_r)\,. \tag{1}$$

The selected decisions determine how the set of trajectories evolves. They may extend a trajectory, introduce an alternative, consolidate evidence across trajectories, resume from an earlier candidate, or terminate the run. The number of candidate-producing decisions in a round determines how many trajectories are advanced concurrently, while successive extensions of the same trajectory determine its iteration depth. Scheduling therefore controls both which trajectories receive work and how much test-time compute they consume.

A static workflow fixes its parallel generation and iteration depth before task-specific evidence is available. Conditioning $\mathcal{A}_r$ on $\mathcal{S}_r$ instead allows the scheduler to adjust both after observing progress, reducing unnecessary candidate generation and the associated EDA feedback calls. This formulation defines the EDA-informed decision interface used by our agents; it does not introduce a separate optimization objective.

4 SMART: EDA-Informed Online Trajectory Scheduling

4.1 Design overview

SMART organizes multi-agent RTL generation as feedback-conditioned scheduling over RTL trajectories. Two principles govern its design. First, feedback from EDA tools updates the allocation of test-time compute: the scheduler determines which trajectories receive further RTL generation and verification and how many are advanced in the next round. Second, candidates and their EDA evidence remain separate by trajectory, allowing the scheduler to compare alternatives or resume from an earlier candidate without merging their debugging histories.

Figure 1 shows the resulting workflow. The planner, RTL generator, feedback reviewer, trajectory comparator, and adaptive trajectory scheduler are implemented as separate LLM-based agents. The planner produces and revises the implementation plan; the RTL generator produces candidates for selected trajectories; the feedback reviewer converts EDA feedback into candidate-level diagnoses; and the trajectory comparator conditionally analyzes evidence from trajectories with competing implementation hypotheses. The native EDA evaluator is not an LLM agent: it compiles and simulates each candidate and returns the resulting feedback.

Given the specification, the current implementation plan, and structured summaries of all evaluated candidates, the adaptive trajectory scheduler determines the next set of trajectory-level decisions: start or continue trajectories, request a trajectory comparison, resume generation from an earlier candidate, or stop with an EDA-verified candidate. Only the scheduler assigns work across trajectories; the other LLM-based agents supply plans, RTL candidates, or analyses without modifying the global schedule.

4.2 Candidate history and EDA evidence

SMART maintains a structured history of the RTL candidates produced during a run. Each candidate record identifies its predecessor, the scheduling decision that produced it, and its verification status. Predecessor links distinguish successive iterations of one trajectory from alternatives initiated from the specification or an earlier candidate. The history gives the scheduler the context needed to compare trajectories and resume generation from any retained candidate.

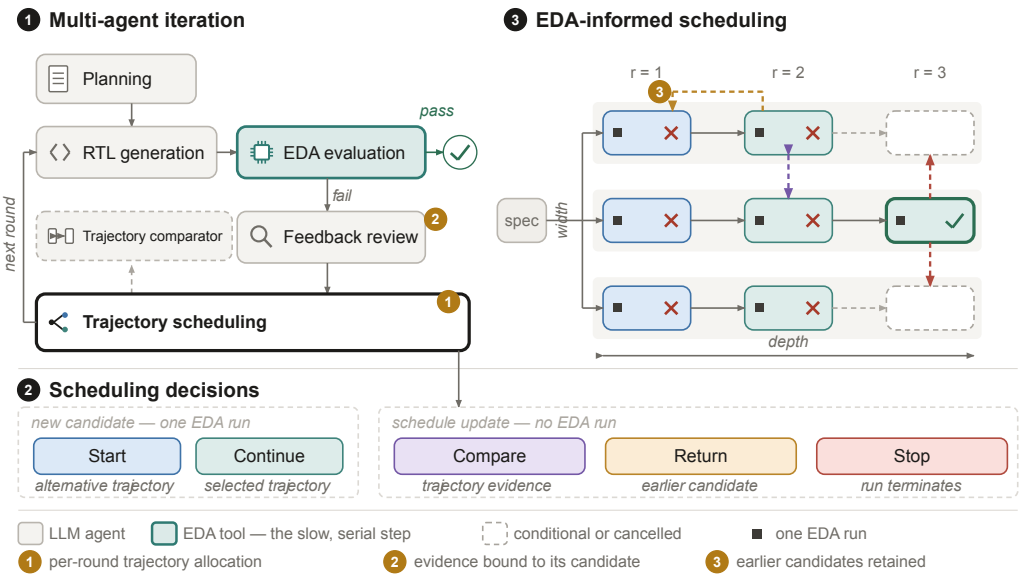


Figure 1: SMART schedules RTL trajectories using feedback from prior EDA evaluations. Candidate generation and evaluation update a structured trajectory history; the adaptive trajectory scheduler then decides which trajectories to start, continue, or compare, whether to return to an earlier candidate, and when to stop. Earlier candidates remain available when a later change regresses.

Candidate-scoped evidence. EDA feedback is attached to the candidate that produced it instead of being appended to a shared multi-agent transcript. The native evaluator records whether the candidate is incomplete, fails compilation, fails simulation, passes, or encounters an infrastructure fault. For a design failure, the feedback reviewer examines the candidate, its change from its parent, and the relevant EDA trace. It identifies the failure, distinguishes newly introduced errors from repeated ones, and summarizes progress or regression relative to the parent. The reviewer neither edits RTL nor selects the next trajectory. This separation lets the scheduler compare evidence across candidates without exposing the generator to unrelated debugging histories.

Retained candidates. Generating a new candidate does not overwrite its predecessor. Earlier candidates and their EDA evidence remain in the history after the scheduler selects another trajectory. If a later change regresses, the scheduler can redirect subsequent work to an earlier candidate while retaining the failed change as evidence. The return decision itself neither generates RTL nor invokes EDA; a new evaluation occurs only if the scheduler subsequently continues that trajectory. The generator and reviewer receive the selected candidate and its relevant local history, whereas the scheduler receives compact summaries for all candidates rather than their RTL source or complete transcripts.

4.3 EDA-feedback-conditioned trajectory scheduling

SMART implements the policy π in Equation 1 with an adaptive trajectory scheduler. At the start of each round, the scheduler reads the specification, the current implementation plan, candidate status and evidence, unresolved implementation hypotheses, and the progress observed in the preceding round. It then emits a variable-length schedule over named trajectories.

The scheduling prompt directs the scheduler to continue a trajectory when EDA feedback localizes the failure and the candidate shows useful progress. It starts an alternative trajectory when the specification supports a materially different implementation hypothesis or when the current direction remains unproductive. A trajectory comparator is invoked only when selected trajectories contain evidence that benefits from direct comparison. It returns a joint analysis to the scheduler without generating RTL, invoking EDA, or choosing the next decision. When a later candidate regresses, the

scheduler may return to an earlier candidate and use the failed attempt to guide subsequent work. It may stop only with a candidate that has passed the native EDA evaluator.

These decisions determine the realized allocation of test-time compute. Starting or continuing several trajectories in one round increases the number of candidate-generation actions that can be executed concurrently; repeatedly continuing the same trajectory increases its iteration depth. Because the scheduler emits a variable number of decisions after observing each round's evidence, neither quantity is fixed for all tasks. Moreover, only starting or continuing a trajectory produces a new candidate and consumes an EDA evaluation. Comparison, return, and stopping change subsequent allocation or terminate the run without an immediate EDA call. The candidate and completed-EDA counts reported in Section 5 therefore measure the realized cost of these scheduling decisions.

4.4 Scheduling execution and validity constraints

Algorithm 1 summarizes the feedback loop. Candidate-producing decisions are dispatched through the worker pool, after which each generated candidate is evaluated by the native EDA flow and added to the candidate history. Failed candidates are reviewed before the next scheduling round. Trajectory comparisons add cross-trajectory evidence, while return decisions select an earlier candidate as the basis for subsequent generation; neither action directly invokes EDA.

Algorithm 1 SMART feedback-conditioned trajectory scheduling loop.

Require: specification S, evaluator EVAL, scheduler π, worker pool P, round cap R
$\mathcal{H} \leftarrow \emptyset$, $\mathcal{E} \leftarrow \emptyset$
for $r = 1, \ldots, R$ **do**
  $\mathcal{A}_r \leftarrow \pi(S, (\mathcal{H}, \mathcal{E}))$
  if $\mathcal{A}_r$ fails deterministic validation **then return** INVALIDDECISION
  if $\textsc{Stop}(c) \in \mathcal{A}_r$ and c is verified **then return** c
  for all candidate-producing $a \in \mathcal{A}_r$, dispatched on P up to capacity **do**
    $c \leftarrow \textsc{Generate}(a)$; $\mathcal{H} \leftarrow \mathcal{H} \cup \{\textsc{Record}(c, a)\}$; $v \leftarrow \textsc{Eval}(c)$; $\mathcal{E} \leftarrow \mathcal{E} \cup \{(c, v)\}$
    if $v \neq \textsc{Pass}$ **then** $\mathcal{E} \leftarrow \mathcal{E} \cup \{(c, \textsc{Review}(c, \text{parent}(c), v))\}$
  Add requested comparisons to $\mathcal{E}$ and retain return targets for the next round
return UNSOLVED

5 Experimental Evaluation

5.1 Experimental setup

Benchmarks and baselines. We evaluate the 34-task agentic Spec2RTL subset of CVDP cid003 [Pinckney et al., 2025] and the 64-task, IP-level ChipVerilog benchmark [Tan et al., 2026]. CVDP provides human-assigned difficulty labels for 5 easy, 25 medium, and 4 hard tasks. All methods use DeepSeek V4 Pro and the native harness of each benchmark. We compare Plain generation, MAGE [Zhao et al., 2024], ACE-RTL [Deng et al., 2026], and SMART; ACE-RTL is the primary baseline for resource comparisons because matched execution telemetry is available for both systems.

Implementation. The planner, trajectory scheduler, RTL generator, feedback reviewer, and trajectory comparator are separate prompted roles. The initial planner uses temperature 0.3, the reviewer uses 0.1, and all other role calls and plan revisions use 0.2. The scheduler observes candidate metadata and summarized EDA evidence. Each evidence record is truncated to 6,000 characters.

Metrics. Pass rate is the fraction of tasks for which the native evaluator returns PASS; we also report the corresponding number of passed tasks. To quantify realized resource use, we report *initiated trajectories*, the number of independent trajectories started from the specification; *candidate evaluation attempts*, the number of generated candidates submitted to validation, including candidates rejected during preflight; and *completed EDA evaluations*, the subset for which the benchmark compiler or simulator returns a result. We additionally report LLM call attempts and total LLM tokens when available. For task groups of different sizes, resource counts are reported as per-task averages.

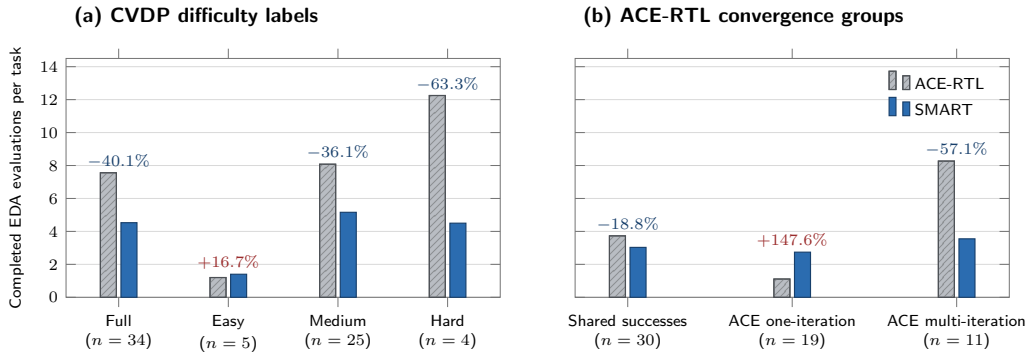


Figure 2: Completed EDA evaluations per task on CVDP. Panel (a) uses the benchmark's difficulty labels; panel (b) groups the 30 tasks passed by both systems according to whether ACE-RTL required one or multiple iterations. Percentages are relative to ACE-RTL.

5.2 Overall pass rate and resource use

Table 1: End-to-end pass rate and resource use. Initiated trajectories are independent trajectories started from the specification. Candidate evaluation attempts include candidates rejected during preflight; completed EDA evaluations count only evaluations that return a compiler or simulator result. Dashes denote unavailable telemetry.

Benchmark	Method	Passed tasks	Pass rate (%)	Initiated trajectories	Candidate eval. attempts	Completed EDA evaluations	LLM call attempts	Total LLM tokens
CVDP cid003	Plain	10/34	29.41	–	–	–	–	–
	MAGE	13/34	38.24	–	–	–	–	–
	ACE-RTL	**32/34**	**94.12**	170	302	257	431	8,510,173
	SMART	31/34	91.18	**71**	**156**	**154**	**322**	**5,496,661**
ChipVerilog	Plain	18/64	28.13	–	–	–	–	–
	MAGE	21/64	32.81	–	–	–	–	–
	ACE-RTL	21/64	32.81	320	853	853	801	17,394,054
	SMART	**22/64**	**34.38**	**78**	**191**	**166**	**741**	**15,314,876**

Table 1 shows similar pass counts with fewer candidate and EDA evaluations. On CVDP, SMART passes 31 tasks versus 32 for ACE-RTL while reducing completed EDA evaluations by 40.1%, initiated trajectories by 58.2%, and total LLM tokens by 35.4%. On ChipVerilog, SMART passes 22 tasks versus 21 for ACE-RTL and reduces completed EDA evaluations by 80.5%, initiated trajectories by 75.6%, and total LLM tokens by 12.0%. These results demonstrate that EDA-informed online scheduling can maintain functional performance while substantially reducing the EDA evaluations required for test-time scaling.

5.3 Completed EDA evaluations across task groups

Figure 2(a) shows that the reduction grows with task difficulty. Relative to ACE-RTL, SMART uses 16.7% more completed EDA evaluations on easy tasks, but 36.1% fewer on medium tasks and 63.3% fewer on hard tasks; the full-subset average falls from 7.56 to 4.53. The hard group contains four tasks, so the medium and full-subset results provide the more stable aggregate evidence.

Panel (b) controls for pass outcome by considering the 30 shared successes. SMART reduces EDA evaluations by 57.1% on the 11 tasks that require multiple ACE-RTL iterations, but increases them from 1.11 to 2.74 per task on the 19 one-iteration tasks. Across all shared successes, initiated trajectories fall by 64.0%, whereas mean successful trajectory depth—the number of candidate-generation levels to the accepted candidate, with the initial candidate at depth one—changes only from 1.87 to 1.97. The reduction therefore comes primarily from avoiding unnecessary trajectories on longer runs; early-converging tasks expose the need for stronger stopping.

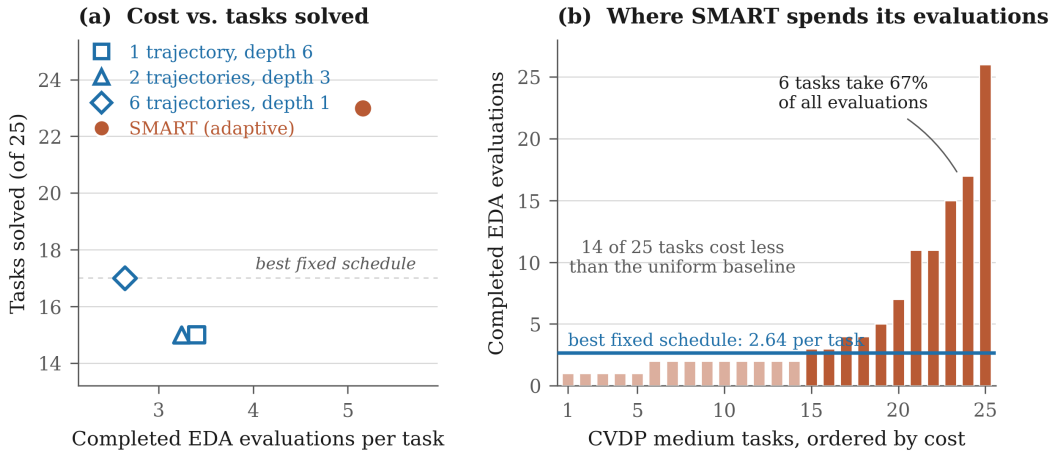


Figure 3: Scheduling-policy ablation on the 25 CVDP medium tasks. Panel (a) compares pass count and realized resource use under a shared six-candidate cap. Panel (b) shows the adaptive scheduler's per-task EDA allocation. Each configuration is a single run.

5.4 Adaptive versus fixed scheduling policies

We isolate scheduling policy on the 25 CVDP medium tasks. All configurations use the same model, prompts, agents, evaluator, worker pool, six-candidate cap, and stop-on-PASS rule. The adaptive scheduler is compared with fixed depth (1×6), fixed width (6×1), and mixed (2×3) schedules; the configurations are matched on candidate cap rather than realized cost.

Figure 3(a) shows that the three fixed schedules solve 15, 15, and 17 tasks, whereas adaptive scheduling solves 21. The fixed schedules also saturate within the cap: pure width gains no pass after its fourth candidate, while six rounds of pure depth improve the pass count only from 12 to 15.

Adaptation spends more in aggregate—5.16 rather than 2.64 EDA evaluations per task for the strongest fixed schedule, with $2.4\times$ as many LLM calls and tokens—but allocates this cost unevenly. Figure 3(b) shows that 14 tasks cost less than 2.64 evaluations, while six tasks account for 67% of all adaptive evaluations. The four additional passes therefore coincide with concentrating work on a small subset rather than applying the same schedule to every task. This single-run ablation is limited to the medium tier, and the 6×1 baseline incurred 43 failed LLM calls. Repairing those calls could improve the fixed-width baseline, so the observed pass-count gap is an upper bound on the advantage of adaptation.

5.5 Case studies

Two task traces connect the aggregate results to the scheduler's trajectory decisions.

Targeted alternatives: `dynamic_equalizer_0001`**.** The complex-LMS specification declares complex I/Q signals without expanding the coefficient update formula, supporting conjugate/cross-coupled and component-wise I/Q interpretations. SMART evaluates both trajectories and stops in round 2 after two completed EDA evaluations. ACE-RTL uses 30 evaluations across the task archive and eight on its passing trajectory. The failed SMART trajectory also contains a fixed-point scaling error, so the trace does not identify the semantic interpretation as the cause of failure. It shows that preserving two specification-grounded alternatives reduced the serial EDA iterations needed to reach a passing design.

Over-expansion: `swizzler_0001`**.** The planner instead treated the weakly supported timing of `parity_error` as an ambiguity and started two trajectories. Both passed because the available tests did not distinguish the interpretations, yet the scheduler continued and completed four EDA evaluations, compared with one for ACE-RTL. Three evaluations occurred after a passing candidate was already available. This trace isolates the complementary failure mode: alternative trajectories

require specification grounding and evaluator-distinguishable hypotheses, and stopping should take priority once a candidate passes.

6 Limitations

Our evaluation uses a single language model, DeepSeek V4 Pro, so the results may not generalize to other models. We also use the number of completed EDA evaluations as the primary measure of EDA cost, without accounting for the different wall-clock costs of compilation, simulation, and design verification. Future work should evaluate SMART across multiple models and measure stage-level runtime and resource costs throughout the EDA flow.

7 Conclusion

We have formulated EDA-grounded multi-agent RTL generation as online scheduling over RTL design trajectories. This formulation replaces prespecified parallel generation and iteration depth with feedback-conditioned decisions about which trajectories to start, continue, or compare, whether to return to an earlier candidate, and when to stop. We have presented SMART, which combines an adaptive trajectory scheduler with candidate-specific evidence, conditional trajectory comparison, and retained candidates. On CVDP and ChipVerilog, SMART achieved pass rates comparable to ACE-RTL while reducing completed EDA evaluations by 40.1% and 80.5%, respectively; on CVDP, it also used 35.4% fewer total LLM tokens. The scheduling ablation showed that no tested fixed width–depth shape reached the adaptive pass count under a shared candidate cap, while trace analysis showed that the reduction came primarily from avoiding unnecessary trajectories on longer runs. Early-converging tasks remain an opportunity for improved stopping. These results support a practical direction for RTL test-time scaling: additional computation should be scheduled according to observed EDA evidence rather than applied through the same fixed loop to every task.

References

J. C.-Y. Chen, S. Saha, and M. Bansal. ReConcile: Round-table conference improves reasoning via consensus among diverse LLMs. *arXiv preprint arXiv:2309.13007*, 2023. URL https://arxiv.org/abs/2309.13007.

W. Chen, J. Yuan, C. Qian, et al. Optima: Optimizing effectiveness and efficiency for LLM-based multi-agent system. *arXiv preprint arXiv:2410.08115*, 2024. URL https://arxiv.org/abs/2410.08115.

C. Deng, Z. Yu, G.-T. Liu, et al. ACE-RTL: When agentic context evolution meets RTL-specialized LLMs. *arXiv preprint arXiv:2602.10218*, 2026. URL https://arxiv.org/abs/2602.10218.

S. Eo, H. Moon, E. H. Zi, et al. Debate only when necessary: Adaptive multiagent collaboration for efficient LLM reasoning. *arXiv preprint arXiv:2504.05047*, 2025. URL https://arxiv.org/abs/2504.05047.

C. Eryilmaz. ChipCraftBrain: Validation-first RTL generation via multi-agent orchestration. *arXiv preprint arXiv:2604.19856*, 2026. URL https://arxiv.org/abs/2604.19856.

W. Fang, L. Yuan, G. Lan, et al. Iterative critique-and-routing controller for multi-agent systems with heterogeneous LLMs. *arXiv preprint arXiv:2605.08686*, 2026. URL https://arxiv.org/abs/2605.08686.

A. Fourney, G. Bansal, H. Mozannar, et al. Magentic-One: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*, 2024. URL https://arxiv.org/abs/2411.04468.

C.-T. Ho, H. Ren, and B. Khailany. VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool. *arXiv preprint arXiv:2408.08927*, 2024. URL https://arxiv.org/abs/2408.08927.

S. Li, A. M. Abdelmoniem, and S. Wang. ProgRouter: Online progress-guided orchestration for multi-agent LLM workflows under quality-cost tradeoffs. *arXiv preprint arXiv:2608.25992*, 2026. URL https://arxiv.org/abs/2608.25992.

B. Niu, Y. Song, K. Lian, et al. Flow: Modularized agentic workflow automation. In *Proceedings of the International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/ba84da6921f3040b74ee163aa7451f53-Abstract-Conference.html.

N. Pinckney, C. Deng, C.-T. Ho, et al. Comprehensive Verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on RTL design and verification. *arXiv preprint arXiv:2506.14074*, 2025. URL https://arxiv.org/abs/2506.14074.

C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *Proceedings of the International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b623663fd9b874366f3ce019fdfdd44-Abstract-Conference.html.

J. Su, Q. Lan, Y. Xia, et al. Difficulty-aware agentic orchestration for query-specific multi-agent workflows. *arXiv preprint arXiv:2509.11079*, 2025. URL https://arxiv.org/abs/2509.11079.

Y. Tan, J. Du, X. Meng, et al. ChipVerilog: A large-scale OpenCores-derived benchmark for LLM-based Verilog RTL generation. *arXiv preprint arXiv:2607.13079*, 2026. URL https://arxiv.org/abs/2607.13079.

S. Thakur, J. Blocklove, H. Pearce, et al. AutoChip: Automating HDL generation using LLM feedback. *arXiv preprint arXiv:2311.04887*, 2023. URL https://arxiv.org/abs/2311.04887.

B. Wang, Q. Xiong, Z. Xiang, et al. RTLSquad: Multi-agent based interpretable RTL design. *arXiv preprint arXiv:2501.05470*, 2025. URL https://arxiv.org/abs/2501.05470.

X. Wang, Z. Liu, S. Feng, et al. On time, within budget: Constraint-driven online resource allocation for agentic workflows. *arXiv preprint arXiv:2605.06110*, 2026. URL https://arxiv.org/abs/2605.06110.

C. Yu, C. Deng, N. Pinckney, and B. Khailany. Agentic hardware design as repository-level code evolution. *arXiv preprint arXiv:2606.28279*, 2026. URL https://arxiv.org/abs/2606.28279.

G. Zhang, L. Niu, J. Fang, et al. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025a. URL https://arxiv.org/abs/2502.04180.

J. Zhang, J. Xiang, Z. Yu, et al. AFlow: Automating agentic workflow generation. In *Proceedings of the International Conference on Learning Representations*, 2025b. URL https://arxiv.org/abs/2410.10762.

Y. Zhao, H. Zhang, H. Huang, et al. MAGE: A multi-agent engine for automated RTL code generation. *arXiv preprint arXiv:2412.07822*, 2024. URL https://arxiv.org/abs/2412.07822.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: Sections 1 and 5 state and evaluate the main claims.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 discusses scope, failure modes, and evaluation limits.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

Justification: The paper presents no theoretical results requiring proof.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Exact prompts, source code are provided if paper accepted.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Exact prompts, source code are provided if paper accepted.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 5.1 specifies the benchmarks, model settings, protocol, and metrics.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We provide experiemnt results and ablation studies.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: The experiments does not rely on compute resources as we use LLM API.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics https://neurips.cc/public/EthicsGuidelines?

   Answer: [Yes]

   Justification: The authors reviewed the NeurIPS Code of Ethics and found no deviations.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: The paper has no negative societal impacts.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The submission releases no high-risk model or dataset.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [No]

    Justification: xisting benchmarks and systems are cited, but their licenses and terms are not stated.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: No new dataset, model, or code asset is released.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The work does not involve crowdsourcing or human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The work does not involve human-subject research.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Sections 5.1 describe the core LLM roles and settings.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.